Programare Web / Dezvoltarea Aplicațiilor Web
Tehnologia Informației, anul IV · Informatică, anul II · Anul universitar 2026–2027, semestrul I

LABORATORUL 02

HTML5 și CSS3: structură semantică, Flexbox, Grid și design responsiv

Construim pagina de prezentare a unui mic magazin, cu elemente semantice, un stil extern bine organizat și o așezare care se adaptează de la telefon la desktop.

Tema	Temele 2–4 din planul disciplinei (examene.pe.ub.ro): HTML5, CSS3, așezarea în pagină și designul responsiv
Durata	2 ore
Aveți nevoie de	proiectul `daw-template-ti-info-2026-2027` clonat (laboratorul 01), editorul și un browser; în VS Code, extensia **Live Server** (PhpStorm și WebStorm au serverul integrat)
La final aveți	folderul `exercitii/lab02/` cu `index.html` și `style.css`, salvat pe GitHub pe branch-ul vostru

1. Obiective

La finalul laboratorului veți putea:

- structura o pagină cu elementele semantice HTML5 și explica de ce contează;
- scrie un formular simplu, accesibil, cu etichete legate de câmpuri;
- organiza stilul într-un fișier CSS extern, cu variabile;
- așeza elementele cu Flexbox (pe un rând) și cu Grid (pe rânduri și coloane);
- adapta pagina la ecrane mici și mari, pornind de la mobil (*mobile first*).

2. Pe scurt: structură, prezentare, adaptare

HTML descrie **ce este** fiecare parte a paginii, iar CSS descrie **cum arată** [1], [2]. Elementele semantice (`<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>`) spun browserului, motoarelor de căutare și cititoarelor de ecran rolul fiecărei zone. Un nevăzător poate sări direct la `<main>` sau la meniu, iar un motor de căutare înțelege ce este conținut principal și ce este secundar. Un `<div>` nu transmite nimic din toate acestea.

Pentru așezare folosim două instrumente moderne [3]. **Flexbox** aranjează elemente pe o singură direcție (un meniu, un rând de butoane). **Grid** aranjează pe două direcții (o grilă de carduri). Designul **responsiv** înseamnă o singură pagină care arată bine pe orice ecran: scriem întâi stilul pentru telefon, apoi, cu `@media (min-width: ...)`, îl îmbogățim pentru ecrane mai late.

3. Pornirea

1. Porniți **Docker Desktop**. Deschideți folderul `daw-template-ti-info-2026-2027` în editor și terminalul integrat.
2. Verificați pe ce branch sunteți: `git branch` marchează ramura curentă cu `*` (sau `git status`, pe primul rând: `On branch …`). Doar dacă nu sunteți deja pe branch-ul vostru, treceți pe el; apoi porniți containerele (înlocuiți `popescu.ion`):

```
git branch                # ramura curenta are * in fata
# doar daca NU sunteti deja pe branch-ul vostru:
git checkout popescu.ion   # sau: git switch popescu.ion
docker compose -f backend/compose.yaml up -d
docker compose -f frontend/compose.yaml up -d
```

Sfat. Pentru paginile de astăzi containerele nu sunt strict necesare, dar păstrăm același ritual la fiecare laborator. Dacă ceva nu pornește, consultați metodologia [6].

3.1 De ce lucrăm în afara aplicației React

Aplicația React din `frontend/` este construită tot din HTML, CSS și JavaScript, dar le ascunde în spatele unor noțiuni noi: componente, JSX, stare. Ca să înțelegeți ce face React în locul vostru, scrieți întâi aceleași lucruri „de mână”, în pagini simple, fără framework. Trecerea se face treptat:

Laboratorul	Ce învățăm	Unde lucrăm
02	structura (HTML5) și aspectul (CSS3) unei pagini	`exercitii/lab02/`
03	comportamentul (JavaScript): DOM, evenimente, date de la un API	`exercitii/lab03/`
04 și următoarele	aceleași idei, organizate în componente React	`frontend/app/src/`

Paginile de exercițiu stau separat de aplicație, ca să nu o modifice și să nu o strice, dar în același repository, pe branch-ul vostru, ca să fie salvate pe GitHub împreună cu restul lucrului.

3.2 Unde creați fișierele

În **rădăcina proiectului** (folderul principal, lângă `backend`, `frontend` și `postman`) creați folderul `exercitii`, iar în el subfolderul `lab02`, cu fișierele `index.html` și `style.css`:

```
daw-template-ti-info-2026-2027/
├── backend/
├── frontend/
├── postman/
└── exercitii/
    ├── lab02/    index.html, style.css
    └── lab03/    index.html, style.css, script.js
```

Folderele se creează cu click dreapta în panoul de fișiere al editorului (**New Folder** / **New → Directory**) sau din terminalul integrat, care este deja în rădăcina proiectului: `mkdir -p exercitii/lab02`.

3.3 Cum deschideți pagina în browser

Paginile din `exercitii/` sunt fișiere statice: nu au nevoie de Docker, ci doar de un mic server web local, care le deschide în browser și le reîncarcă după fiecare salvare.

Editor	Cum deschideți pagina	Adresa din browser
VS Code	instalați o singură dată extensia **Live Server** (Extensions → căutați „Live Server”, autor Ritwick Dey → Install); apoi click dreapta pe `index.html` → **Open with Live Server** (sau butonul **Go Live** din bara de jos). Pagina se reîncarcă singură la fiecare salvare.	`http://127.0.0.1:5500/exercitii/lab02/index.html`

Editor	Cum deschideți pagina	Adresa din browser
PhpStorm / WebStorm	au serverul integrat, fără extensie: deschideți `index.html` și duceți mouse-ul în colțul din dreapta-sus al editorului, unde apar pictogramele browserelor, apoi alegeți browserul (sau click dreapta pe fișier → **Open In → Browser**). Pentru reîncărcare automată: **Settings → Tools → Web Browsers and Preview → Reload page in browser: On Save**.	`http://localhost:63342/daw-template-ti-info-2026-2027/exercitii/lab02/index.html`

> **Sfat.** Dacă pagina nu se actualizează, salvați fișierul (**Ctrl + S**, pe macOS **Cmd + S**) și reîncărcați manual (**F5**). Deschiderea cu dublu-click pe fișier funcționează și ea, dar fără reîncărcare automată.

4. Structura semantică (HTML5)

> **Unde.** Fișierul `exercitii/lab02/index.html`, creat la secțiunea 3.2, în rădăcina proiectului (nu în `frontend`).

1. Scrieți în `index.html` începutul paginii: antetul documentului, apoi `<header>` cu meniul:

```
<!DOCTYPE html>
<html lang="ro">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Librăria din cartier — cărți și noutăți</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="antet">
    <a class="logo" href="#">Librăria din cartier</a>
    <nav aria-label="Meniul principal">
      <ul class="meniu">
        <li><a href="#produse">Produse</a></li>
        <li><a href="#despre">Despre noi</a></li>
        <li><a href="#newsletter">Newsletter</a></li>
      </ul>
    </nav>
  </header>
```

2. Continuați, sub `</header>`, cu conținutul principal: produsele, ca articole, și o zonă laterală:

```
    <main class="continut">
      <section id="produse">
        <h1>Recomandările săptămânii</h1>
        <div class="grila">
          <article class="card">
            <h2>Amintiri din copilărie</h2>
            <p>Ion Creangă. Ediție ilustrată.</p>
            <p class="pret">45 lei</p>
            <button type="button" class="buton">Adaugă în coș</button>
          </article>
          <article class="card">
            <h2>Poezii</h2>
            <p>Mihai Eminescu. Antologie pentru liceu.</p>
            <p class="pret">32 lei</p>
            <button type="button" class="buton">Adaugă în coș</button>
          </article>
        </div>
      </section>
      <aside id="despre">
        <h2>Despre noi</h2>
        <p>O librărie mică, deschisă zilnic între 9 și 19, cu livrare în oraș.</p>
      </aside>
```

3. Încheiați cu formularul de abonare, subsolul și etichetele de închidere:

```
      <section id="newsletter" class="newsletter">
        <h2>Abonați-vă la newsletter</h2>
        <form>
          <label for="email">Adresa de e-mail</label>
          <input id="email" name="email" type="email" required>
          <button type="submit" class="buton">Mă abonez</button>
        </form>
      </section>
    </main>
    <footer class="subsol">
      <p>&copy; 2026 Librăria din cartier · exercițiu de laborator PW/DAW</p>
    </footer>
  </body>
  </html>
```

4. Salvați și deschideți pagina în browser, ca la secțiunea 3.3: **Open with Live Server** în VS Code sau pictograma browserului din colțul din dreapta-sus în PhpStorm/WebStorm.

Rezultat. Pagina apare ca text simplu, negru pe alb, încă fără stil. Apăsați **Tab** de câteva ori: focalizarea trece prin legături, câmp și butoane. Apăsați **Mă abonez** cu câmpul gol: browserul cere o adresă, datorită lui `type="email"` și `required`.

De ce. `<label for="email">` este legat de `<input id="email">`: un click pe etichetă activează câmpul, iar cititorul de ecran anunță ce trebuie introdus. Atributul `aria-label` dă un nume meniului pentru tehnologiile de asistare.

5. Stilul (CSS3): variabile, Flexbox, Grid, @media

1. Scrieți în `style.css` variabilele și stilul general:

```
:root {
  --culoare-principala: #1f3a73;
  --culoare-accent: #b45309;
  --fundal: #f6f7fb;
  --text: #1f2937;
  --raza: 8px;
}
* { box-sizing: border-box; }
body {
  margin: 0; line-height: 1.5;
  font-family: system-ui, Arial, sans-serif;
  color: var(--text); background: var(--fundal);
}
```

2. Adăugați antetul cu meniul (Flexbox) și grila de carduri (Grid), gândite întâi pentru telefon:

```
.antet {
  display: flex; flex-direction: column; align-items: center; gap: 0.5rem;
  padding: 1rem; background: var(--culoare-principala);
}
.logo { font-size: 1.4rem; font-weight: bold; }
.logo, .meniu a { color: #fff; text-decoration: none; }
.meniu { display: flex; gap: 1rem; margin: 0; padding: 0; }
.meniu li { list-style: none; }
.meniu a:hover { text-decoration: underline; }

.continut {
  display: grid; gap: 1.5rem;
  max-width: 1100px; margin: 0 auto; padding: 1rem;
}
.grila { display: grid; grid-template-columns: 1fr; gap: 1rem; }
.card {
  background: #fff; border-radius: var(--raza); padding: 1rem;
  box-shadow: 0 1px 4px rgba(0, 0, 0, 0.12);
}
.pret { font-weight: bold; color: var(--culoare-principala); }
```

3. Adăugați butoanele (cu efect la trecerea mouse-ului), formularul și subsolul:

```
.buton {
  background: var(--culoare-accent); color: #fff;
  border: none; border-radius: var(--raza);
  padding: 0.6rem 1rem; cursor: pointer;
  transition: background 0.2s;
}
.buton:hover, .buton:focus-visible { background: #92400e; }
.newsletter form {
  display: flex; flex-direction: column; gap: 0.5rem; max-width: 420px;
}
.newsletter input { padding: 0.6rem; border: 1px solid #9ca3af; }
.subsol {
  text-align: center; padding: 1rem;
  color: #fff; background: var(--culoare-principala);
}
```

4. La sfârșitul fișierului, adăugați regulile pentru ecrane mai late:

```
/* Tabletă: de la 600px în sus */
@media (min-width: 600px) {
  .antet { flex-direction: row; justify-content: space-between; }
  .grila { grid-template-columns: repeat(2, 1fr); }
}
/* Desktop: de la 900px în sus */
@media (min-width: 900px) {
  .continut { grid-template-columns: 3fr 1fr; }
  .newsletter { grid-column: 1 / -1; }
  .grila { grid-template-columns: repeat(3, 1fr); }
}
```

5. Salvați și deschideți instrumentele dezvoltatorului (**F12**). Apăsați pictograma *Toggle device toolbar* (telefon și tabletă) și modificați lățimea ecranului.

Rezultat. Pe telefon, cardurile sunt unul sub altul, iar logo-ul stă deasupra meniului. Peste 600px apar două carduri pe rând, iar antetul ocupă un singur rând. Peste 900px, grila are trei coloane, „Despre noi” trece în dreapta, iar newsletter-ul ocupă toată lățimea. Butoanele se închid la culoare la trecerea mouse-ului. În cod, `1fr` înseamnă „o parte din spațiul liber”, iar `grid-column: 1 / -1` întinde elementul pe toată lățimea grilei.

6. Exerciții

1. Adăugați încă patru carduri cu produse. Verificați că grila rămâne ordonată pe toate lățimile.
2. Schimbați paleta modificând doar variabilele din `:root`. Verificați contrastul textului alb de pe butoane cu WebAIM Contrast Checker.
3. Adăugați în `<footer>` un al doilea paragraf cu adresa și programul, iar în CSS așezați cele două paragrafe unul lângă altul, cu Flexbox, doar de la 600px în sus.
4. ★ (opțional) Adăugați fiecărui card o imagine (`<img>` cu `alt` descriptiv) și limitați-i înălțimea cu `height: 160px; object-fit: cover;`.
5. ★ (opțional) Înlocuiți cele două reguli `@media` ale grilei cu una singură: `grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));`. Explicați ce se schimbă.

7. Salvarea lucrului și oprirea

```
git status
git add .
git commit -m "Laboratorul 02: pagina de prezentare a magazinului"
git push origin popescu.ion
# sau doar: git push (dupa ce ati facut o data git push -u)
docker compose -f backend/compose.yaml stop
docker compose -f frontend/compose.yaml stop
```

Push. `git push origin popescu.ion` este forma completă: trimite branch-ul vostru la `origin`, adresa proiectului de pe GitHub. Forma scurtă, `git push`, merge doar dacă ați făcut anterior, o dată, `git push -u origin popescu.ion`; `-u` leagă branch-ul local de cel de pe GitHub.

Rezultat. `git status` arată, înainte de commit, folderul nou `exercitii/`. După `git push`, folderul apare pe GitHub, pe branch-ul vostru.

8. Ce arătați la final

- pagina deschisă în browser, la trei lățimi (telefon, tabletă, desktop);
- în codul HTML, elementele semantice și eticheta legată de câmpul de e-mail;
- exercițiile 1–3 rezolvate și commit-ul vizibil pe GitHub, pe branch-ul vostru.

Resurse pentru exersare

- **W3Schools – elemente semantice HTML5:** https://www.w3schools.com/html/html5_semantic_elements.asp
- **W3Schools – Flexbox:** https://www.w3schools.com/css/css3_flexbox.asp
- **W3Schools – Grid:** https://www.w3schools.com/css/css_grid.asp
- **W3Schools – design responsiv:** https://www.w3schools.com/css/css_rwd_intro.asp
- **Curs HTML și CSS în limba română (playlist YouTube):** https://www.youtube.com/playlist?list=PL7aWL1nkGk0zssBjfDBtRMxtAGL3KNmAJ
- **MDN – structurarea conținutului cu HTML:** https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content

Bibliografie

[1] E. Pruteanu și I. Furdu, *Modele, tehnologii, servicii și aplicații integrate de tip R.I.A. pentru web*. Bacău: Alma Mater, 2022.

[2] S. Buraga, *Tehnologii Web*. București: Matrix Rom, 2001.

[3] Mozilla, *MDN Web Docs – CSS layout*. developer.mozilla.org

[4] W3C, *Web Content Accessibility Guidelines (WCAG) 2.2*, 2023. w3.org/TR/WCAG22

[5] C.-M. Papară, *Metodologie de lucru și template de proiect – Programare Web / Dezvoltarea Aplicațiilor Web*, UVAB, 2026.